

Birla Institute of Technology & Science, Pilani Hyderabad Campus**CS F372: Operating Systems (Assignment 2)****Assigned: 26/02/2026****Date of Submission: 21/03/2026****Total Marks: 16****Weightage: 8%****General Instructions:**

- This assignment can be done in groups of **maximum 4** students.
- All your programs must run on a **Ubuntu 24.X virtual machine on Oracle Virtual Box**.
- Recommended allocation: 4 cores, 50 GB virtual disk, 4GB RAM or more on VM.
- Submit all source files and executables along with a readme that has group details (Names and ID numbers of all members of the group.)
- Put all your deliverables in a zip/tar file and submit through course webpage (assignment section). One submission per group is sufficient.
- Every assignment will have a demo and viva, the schedule of which will be intimated later through a separate notice.

For any assignment related queries, you may contact I/C or the following teaching assistants:

- Arpit Kudre <f20230158@hyderabad.bits-pilani.ac.in>
- Parth Munjal Joshi <f20230127@hyderabad.bits-pilani.ac.in>

Q1. (4 Marks)**Multithreaded Statistical Sorting Application****Objective:**

Write a multi-threaded POSIX compliant C program that performs parallel sorting (in ascending order) with statistical analysis.

Input and Array:

1. There exists a global array of integers of size 2^n where $2 \leq n \leq 10$.
2. The global array should be initialized with random integers of your choice in the range [0,99] as shown in the sample testcase below.
3. The program takes an input, say m , from the user where $2 \leq m \leq n$ and it is also a power of 2.

Threading Model:Worker Threads:

Each of the worker threads must:

1. Sort its assigned elements of the global array in-place with any sorting algorithm of your choice. They must not modify elements beyond their assigned segments.
2. Compute the following statistics for its assigned segment
 - a) Average: Arithmetic mean of all elements in the segment
 - b) Maximum: Largest element in the segment
 - c) Minimum: Smallest element in the segment
3. Store these results in a struct. The struct parameters are shown below.

Allocate the thread result struct using **malloc()**.

```

struct thread_result {
    int thread_id;           // Logical thread ID
    double average;          // Average of the segment
    int minimum;             // Smallest Element
    int maximum;             // Largest Element
};

```

4. Terminate after completing its work and pass the struct pointer to the parent via **pthread_exit()**.

Parent Thread:

The main parent thread must:

1. Create the m worker threads and pass the following struct as arguments.

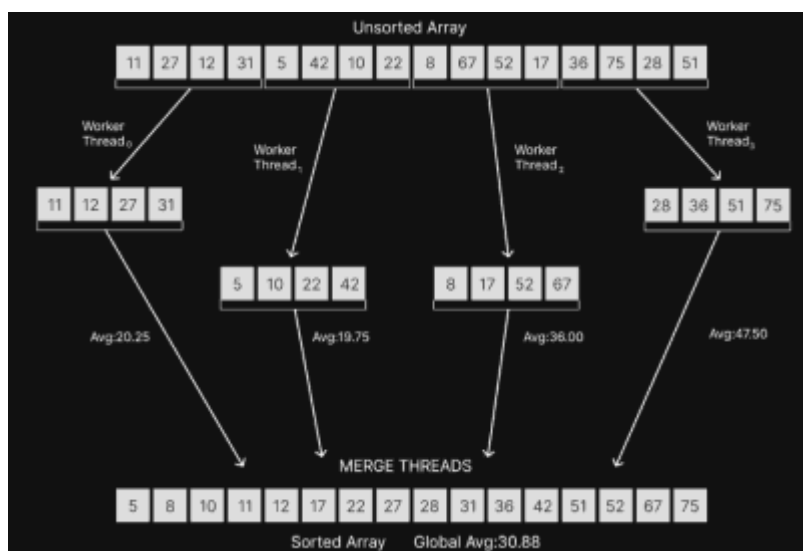
```

struct thread_params {
    int thread_id;           // Logical thread ID
    int start_index;         // Starting index in global_array
    int segment_size;        // Number of elements to process
};

```

2. Wait for all the threads to complete and capture returned pointers via **pthread_join()**.
3. Aggregate the statistics across all the segments
 - a) Global average
 - b) Global maximum
 - c) Global minimum
4. Perform a **k-way merge** of the m sorted segments into a new global array of the same size.
5. Display the results in the following order:
 - a) Statistics of each individual thread ordered by their **thread_id**
 - b) The aggregated (global) statistics
 - c) The final sorted array

Sample Testcase:



Sample Output:

```

Enter the number of segments
4
Initial Unsorted Array: 11 27 12 31 5 42 10 22 8 67 52 17 36 75 28 51

--- Worker Thread Stats ---
Thread 0: Avg: 20.25 | Min: 11 | Max: 31
Thread 1: Avg: 19.75 | Min: 5 | Max: 42
Thread 2: Avg: 36.00 | Min: 8 | Max: 67
Thread 3: Avg: 47.50 | Min: 28 | Max: 75

--- Global Statistics ---
Global Average: 30.88
Global Minimum: 5
Global Maximum: 75

Final Sorted Array: 5 8 10 11 12 17 22 27 28 31 36 42 51 52 67 75

```

NOTE:

- You may use `pthread_create()`, `pthread_join()`, `pthread_exit()`, `pthread_self()`, `pthread_detach()`, and `pthread_cancel()` to implement this.
- Do NOT use any synchronization primitives such as `pthread_mutex_lock()`, `pthread_mutex_unlock()`, semaphores (`sem_wait`, `sem_post`), or condition variables.

Q2. (5 Marks)**Real-Time Round-Robin Process Scheduling:****Objective:**

To observe the relationship between the time quantum, preemption and context switching under CPU-Bound and I/O-bound workload characteristics.

The Question:

- Preparation: Modify the system-wide round-robin timeslice to 5 different values: 2ms, 5ms, 20ms, 100ms and 200ms.
HINT: You will need to change the `sched_rr_timeslice_ms` kernel parameter.
- Implementation: Write two C programs. Each program must fork 8 child processes.
 - Program A (CPU-Bound): Each child process must execute the task provided in `cpu_bound.h`.
 - Program B (I/O-Bound): Each child process must execute the task provided in `io_bound.h`.
- Execution: To ensure valid execution, you must force all 8 processes to compete for the **same CPU core**. If they run on different cores, they will not preempt each other frequently leading to incorrect or misleading results. Execute your programs with round-robin scheduling using the **SCHED_RR** real-time policy.

Execution command: `sudo taskset -c 0 chrt -r <priority> ./program_name`

Explanation:

- `taskset -c 0` : Pins all processes to CPU core 0

- `chrt -r <priority>` : Sets SCHED_RR scheduling policy with priority (recommended: 10-50)

- **Data Collection and Submission:** For each of the time quantum values and for both programs, record the following output:
 - Involuntary Context Switches (aggregated across all children)
 - Voluntary Context Switches (aggregated across all children)
 - Total Elapsed Time (measured by the parent process)

Your parent program may use the `getrusage(RUSAGE_CHILDREN, &usage)` to retrieve aggregated context switch information for all terminated child processes. Make sure to submit a **.txt file** along with the C programs containing the aforementioned statistics.

Sample Text File:

```
Program-A (CPU-Bound)
-----
Time Quantum: 2ms
Involuntary Context Switches: 1742
Voluntary Context Switches: 8
Total Elapsed Time: 4.23s

[....continue for all the time quantum values....]

Program-B (I/O-Bound)
-----
[Same format as above]
```

Relevant Links:

[Getrusage linux manpage](#)
[Tuning scheduling policy](#)
[Taskset linux manpage](#)
[#2 Syntax Time Command Output](#)
[Round Robin Scheduling](#)

Q3. (4 Marks)

Sudoku Validator and Analytics Engine:

Objective:

This problem requires you to design a multi-threaded POSIX compliant C program that validates a given 9x9 grid sudoku solution and produces certain analytics in case of incorrectness.

Rules of the Game:

Every cell must be filled with a number between 1 and 9. Every horizontal row must have every number from 1 to 9 exactly once. Every vertical column must have every number from 1 to 9 exactly once. Each of the nine 3x3 sub-grids must contain every number from 1 to 9 exactly once.

Validation Phase:

The program involves two phases the validation phase and the analytics phase. During the validation phase the parent spawns 27 worker threads. The worker threads are divided as follows: 9 threads for rows, 9 threads for columns and 9 threads for sub-grids.

The parent thread waits for the validation threads to complete. The validity of the sudoku is tracked through a 27 sized global validity array, where each of the 27 worker threads write their validity status (VALID = 1) and (INVALID = 0). It is important that each thread only writes to its own specific index to

avoid any race conditions. Once all the validation worker threads are done the parent thread goes over the validity array and prints "VALID" or "INVALID" based on the validity of the sudoku.

Analytics Phase:

Once the validation phase is over, if the sudoku is VALID the program can exit but, if it is INVALID the program enters the analytics phase to quantify the amount and location of the errors. *Validity Score:* First the parent thread calculates the percentage of threads (out of 27) that returned valid as their result and prints this value as the validity score.

Error Heatmap: Then the parent thread spawns 9 worker threads to fill in an error heatmap. The error heatmap is a 9x9 grid, just like the sudoku, each cell in this heatmap corresponds to each cell of the sudoku. The heatmap provides a way to visualize which cells are contributing to violations in the sudoku rules and to what extent. The 9 worker threads for this error heatmap are initialized for each row. The error scores in the heatmap are populated as follows:

Error Score of a cell = (Error Status of row) + (Error Status of column) + (Error Status of sub-grid)

Where, error status of a row/column/sub-grid is 1 if it was INVALID. (Say, for the top left cell: it belongs to the 0th row, 0th column and 0th sub-grid so we add the NOT of the validity bits in our validity array for these to get the error score)

Hence, cells with error score 0 are cold and those with error score 3 are hot (suspected epicenters of error). Validity array is treated as read-only with respect to analytics threads to avoid race conditions.

Sample Testcases:

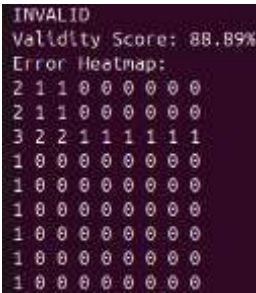
Ip1:

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

 Op1: **VALID**

Ip2:

1	2	3	4	5	6	7	8	9
4	5	6	7	8	9	1	2	3
1	8	9	1	2	3	4	5	6
2	3	4	5	6	7	8	9	1
5	6	7	8	9	1	2	3	4
8	9	1	2	3	4	5	6	7
3	4	5	6	7	8	9	1	2
6	7	8	9	1	2	3	4	5
9	1	2	3	4	5	6	7	8

 Op2: 

NOTE:

- You may use `pthread_create()`, `pthread_join()`, `pthread_exit()`, `pthread_self()`, `pthread_detach()`, and `pthread_cancel()` to implement this.
- Do NOT use any synchronization primitives such as `pthread_mutex_lock()`, `pthread_mutex_unlock()`, semaphores (`sem_wait`, `sem_post`), or condition variables.

Q4. (3 Marks)**Fork Bomb Diffusion & VIP Scheduling:****The Scenario:**

You are a kernel developer for a high-performance compute cluster. The cluster is used by multiple researchers who run experimental workloads. Recently, two critical issues have emerged:

- 1. Malicious Code:** Users occasionally run "fork bombs" that destabilize the entire node.
- 2. Priority Processing:** A specific "VIP" user needs more CPU time than others for urgent workloads.

Your responsibility is to modify the Linux kernel (v6.X) to introduce:

- A fork rate limiter with automatic offender reaping.
- A VIP scheduling mechanism.
- A design where performance privileges never override safety policies.

Objective:

Design and implement two orthogonal kernel-level mechanisms:

1. Fork Bomb Diffusion System

A proactive kernel mechanism that detects excessive process creation and neutralizes the offending workload.

2. VIP User Scheduling Policy

A mechanism to "crown" one user at a time, allowing their tasks to receive preferential CPU scheduling under CFS.

The two mechanisms must coexist without interfering with each other.

Fork Bomb Diffusion Policy Requirements:

Your kernel must:

- Monitor process creation frequency on a per-user basis.
- Detect abnormal fork rates within a configurable time window.
- Throttle excessive fork attempts.
- Automatically terminate the offending process group.
- Restore shell control to the user after fork bomb diffusion.
- Log detection events using kernel logging.
- Exempt the root user to preserve system recovery capability.
- Allow threshold configuration without recompiling the kernel using sysctl.

Fork Bomb Diffusion Expected Behaviour:**On a Stock Kernel:**

- System becomes temporarily unresponsive.
- Fork attempts repeatedly fail.
- User must manually kill the terminal.

On Your Modified Kernel:

- Short pause after detection.
- Offending process group is killed automatically.
- Shell control is restored.
- dmesg shows diagnostic logs.

VIP User Policy Requirements:

Your kernel must:

- Allow root to "crown" exactly one user at a time.
- Crowned user's processes must receive significantly more CPU time.

- Children of a crowned user's processes must inherit VIP status.
- Provide a way to remove VIP status ("uncrown").
- Prevent starvation of non-VIP users.
- Ensure VIP privilege does NOT bypass fork rate limiting.

VIP User Expected Behaviour:

Without VIP:

Two CPU-bound users share CPU approximately equally.

With VIP Crowned:

The crowned user receives a visibly larger share of CPU time.

VIP + Fork Bomb:

If the crowned user runs a fork bomb:

- The rate limiter must still detect and diffuse it.
- Safety policy must override scheduling privilege.

A detailed document on how to accomplish these kernel modifications is attached with this assignment as a separate help doc.

Test Cases:

The system must have at least two users one having root privileges and one being a non-root user.

1. We first run a dummy workload **taskset -c 0 yes > /dev/null &** on both users and monitor cpu usage using **top**. In this baseline a near 50-50 split is expected.
2. We then crown the non-admin user using the admin user terminal and the custom command **crown <non-admin_username>**. We do this while the dummy workload and top terminals are still running and observe crowned user gets a greater CPU share.
3. We revert to fair CPU distribution by testing the uncrowning mechanism **crown reset**.
4. We now test the fork bomb diffusion without a crowned user for this run **sudo forkstat** on the admin terminal then run a fork bomb **:(| :|:&)::** on the non-admin terminal. We expect the fork bomb to stop after the threshold set is reached and the fork bomb to automatically die returning control of that shell back to us. **(If you must manually kill the terminal diffusion has failed)**
5. Now repeat the experiment in the previous point to test fork bomb diffusion but with the non-admin user first crowned before running the fork bomb.
6. Use **dmesg** to check the kernel logs for Fork Bomb and Crown related log messages.

These testcases are also present in the help doc with screenshots for better understanding.

You need not submit anything for this question in the file upload but must demonstrate these kernel modifications successfully during the viva/demo.

DISCLAIMER: While these modifications defuse fork bombs and add VIP users, they introduce significant side effects. Legitimate parallel processes that produce too many fork requests will be unfairly penalized. Modifications to the scheduling hot-path can introduce a slight overhead, potentially decreasing overall system throughput.

-----###-----